

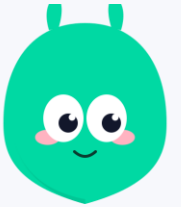
xv6 Process Scheduling

From Theory to Implementation

How xv6 decides which process runs next — from context switches in kernel/proc.c to building your own scheduler.



say hi to xv6-tan!



Where We're Headed

Four parts: quick theory, xv6's data structures, the scheduler's implementation, then we build something.

01



Theory Recap

LDE, dual mode, and traps — the guardrails the kernel relies on.

02



xv6 State Model

struct proc and struct context: what the kernel actually tracks.

03



Scheduling, Deep Dive

swtch(), the scheduler() loop, and the path of preemption.

04



Build & Benchmark

Implement FIFO live, then compare it against Round Robin.

Quick skim — covered in lecture, here as grounding before we open kernel source.



Limited Direct Execution

Programs run directly on the CPU for speed, with the OS installing guardrails so it can regain control.



Dual Modes

User mode restricts what code can do; kernel mode lifts those restrictions. Hardware enforces the boundary.

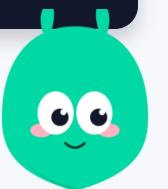
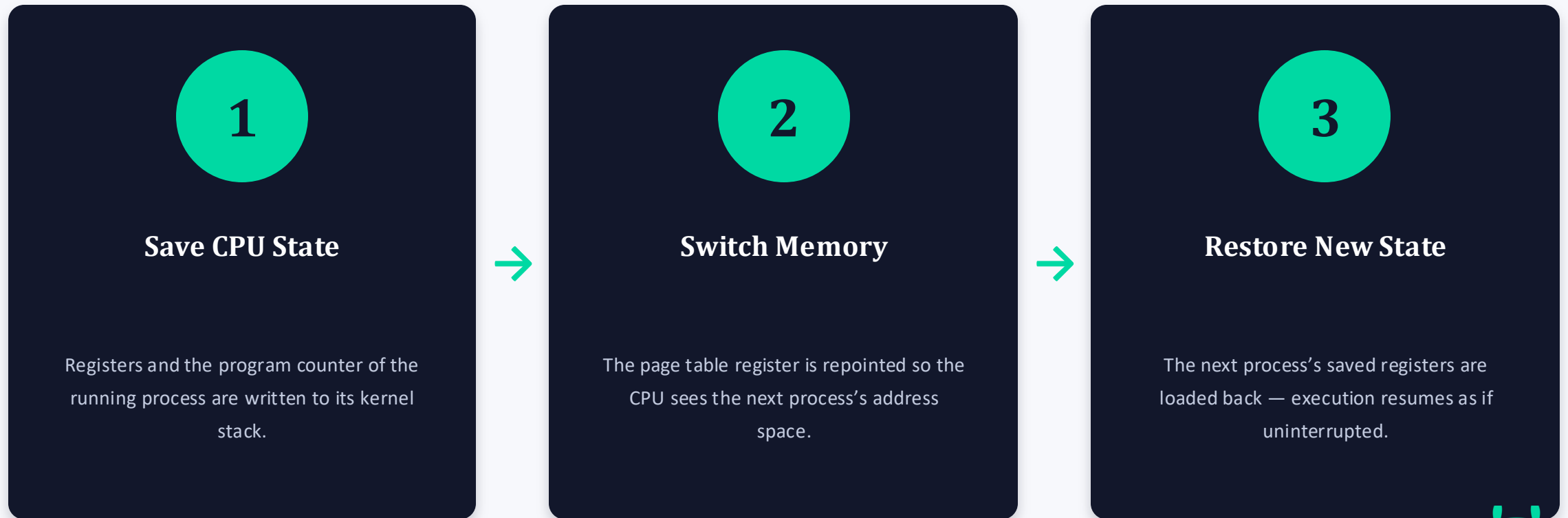


Traps

Interrupts and exceptions are how execution crosses from user mode into the kernel — including the timer.

Context Switch: High-Level View

Saving and restoring process state so one CPU can juggle many processes — the illusion of concurrency.





struct proc

state

RUNNABLE, RUNNING, SLEEPING, ZOMBIE ... the process's current lifecycle stage.

lock

A spinlock guarding this proc's fields — required before touching state.

pid / parent

Identity and family tree — used by fork(), wait(), and our headcount().

pagetable, kstack

Memory and file-descriptor bookkeeping for the whole process.



struct context

ra, sp

Return address and stack pointer — where execution resumes and its stack.

s0 - s11

The callee-saved registers — exactly what switch() must preserve.

Held inside proc

Every struct proc embeds one — its "resume point" when rescheduled.

Not the whole CPU

Caller-saved registers are the compiler's problem, not switch()'s.

PART 3

xv6 Scheduling Implementation

The deep dive — straight into kernel/proc.c and kernel/swtch.S.



The swtch() Function



The Scheduler Loop



The Path of Preemption



The `swtch()` Assembly Function

`kernel/swtch.S` — it doesn't know about processes. It just swaps register sets.

What `swtch()` Does

- 1 Saves the current process's callee-saved registers into its context.
- 2 Loads the next process's registers from its saved context.
- 3 Performs the actual CPU register swap — no C code runs in between.

Critical detail: `swtch()` returns into a different context than it was called from — that's how a suspended process "wakes up" mid-function.

Key Registers Involved

`sp`

Stack pointer — where this process's kernel stack currently sits.

`ra`

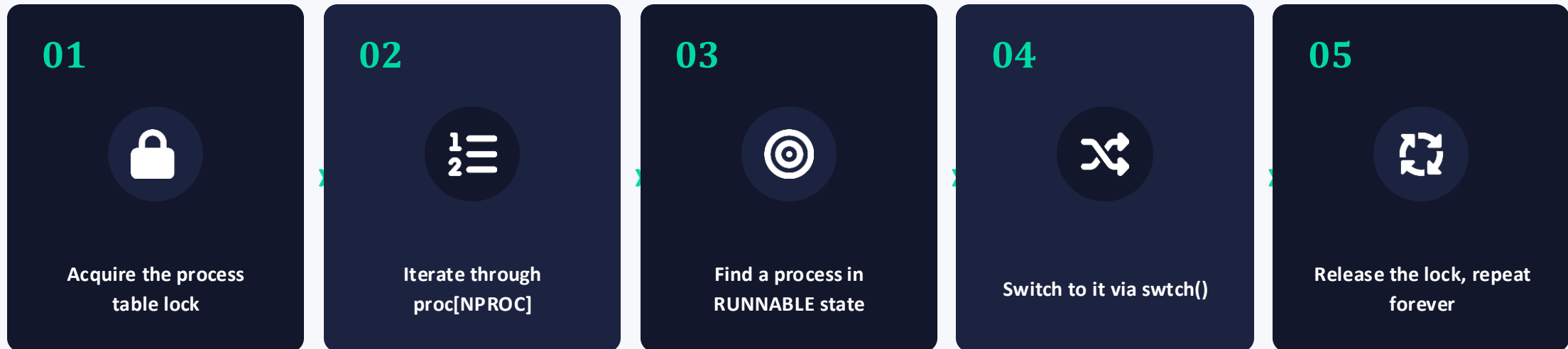
Return address — the instruction to resume at when restored.

`s0 - s11`

The callee-saved registers — everything else is the caller's job.

The Scheduler Loop

`scheduler()` in `kernel/proc.c` — an infinite loop, running on every CPU, that never returns.

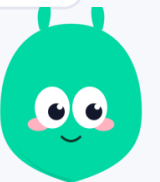
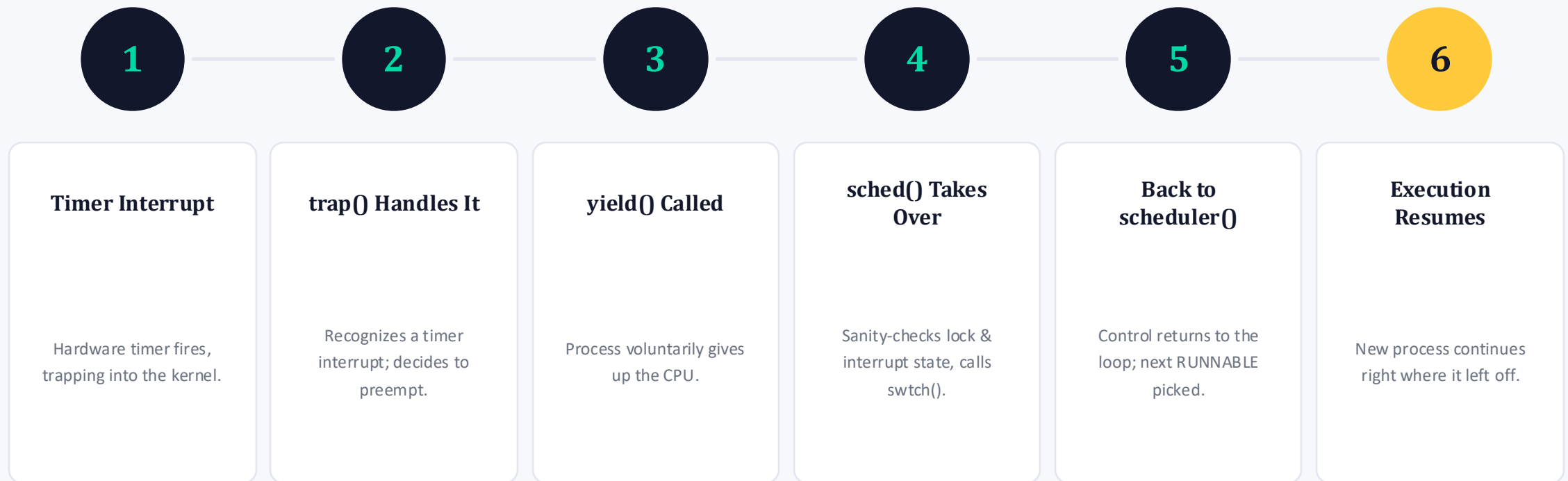


Important Considerations

- Per-process locks prevent races when multiple CPUs scan `proc[]` at once.
- The scheduler itself runs on a dedicated context — it is not "a process."
- Round-robin scanning order gives rough fairness, but no priority or wait-time awareness.

The Path of Preemption

From a hardware timer tick to a different process running — six hops through the kernel.



Practical Demonstration



The Challenge

Implement a FIFO (First-In, First-Out) scheduler — replacing round robin's scan order with strict arrival order by pid.



What You'll Learn

How to modify `scheduler()`, compare pids across `RUNNABLE` processes, and benchmark scheduling policies against each other.

We'll code this together, step by step, then run benchmarks comparing FIFO to Round Robin.



FIFO Scheduler: Implementation Overview

Four steps turn xv6's round robin scheduler into strict first-in, first-out ordering by pid.

1

pid Marks Arrival Order



`allocproc()` hands out pid from `nextpid` in increasing order, so a lower pid means the process arrived earlier.

2

Modify the Scheduler



In `scheduler()`, instead of switching to the first `RUNNABLE` process found, scan the whole table and track the smallest pid.

3

Write a Test Program



`user/fcfs_test.c` forks 3–4 children; each prints its pid in a loop, then exits. Add it to `UPROGS`.

4

Run and Verify



make `qemu CPUS=1` — on one CPU, output order should reveal the lowest-pid process running first.

Performance: FIFO vs Round Robin

Illustrative numbers from a mixed workload — your benchmark run will produce the real ones.

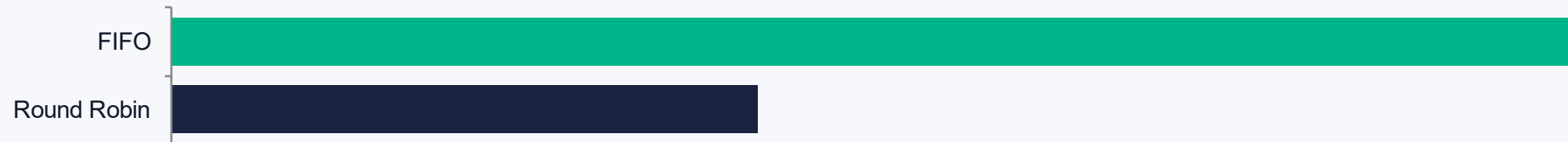
Round Robin (default)

- Fixed time quantum per process
- Simple and fair by construction
- Ignores arrival order — scan position decides who's next

FIFO (ours)

- Lowest pid — earliest arrival — runs first
- Simple and fully predictable ordering
- A long early process can block short later ones (convoy effect)

Average Turnaround Time (subjective)



Key Takeaways

The five ideas worth remembering after today.



Context switching is critical

Efficient saves/restores are what make fair, responsive multitasking possible.



The scheduler is the heart of the OS

It decides which process runs when — every other guarantee builds on that decision.



xv6 is a clean teaching kernel

Small enough to read end-to-end, real enough that the lessons transfer.



Algorithms optimize for different goals

Fairness, throughput, and response time trade off against each other.



You can modify it yourself

With scheduler() understood, custom policies like FIFO are a few dozen lines away.

